Guía de instalación y uso de MPI con Python 3

Miguel Angel Rodríguez Jódar. Dept. Arquitectura y Tecnología de Computadores. Universidad de Sevilla. ©2024.

Instalación del soporte de MPI para Python en GNU/Linux.

Desde un terminal (shell) ejecute el siguiente comando:

```
python3 -m pip install mpi4py
```

El proceso de instalación tiene esta pinta:

```
Looking in indexes: https://pypi.org/simple,
https://www.piwheels.org/simple
Collecting mpi4py
  Downloading
https://files.pythonhosted.org/packages/2e/1a/1393e69df9cf7b04143a517
76727dd048586781bca82543594ab439e2eb4/mpi4py-3.1.5.tar.gz (2.5MB)
    100% |████████████████████████████████| 2.5MB 153kB/s
  Installing build dependencies ... done
Building wheels for collected packages: mpi4py
  Running setup.py bdist_wheel for mpi4py ... done
  Stored in directory:
/home/practica/.cache/pip/wheels/6a/a2/4d/68998a0c10a3a307e55777b41b3
da359a4742f087eff53acce
Successfully built mpi4py
Installing collected packages: mpi4py
Successfully installed mpi4py-3.1.5
```

Si ya estuviera instalado de una sesión anterior, la consola mostrará algo como esto:

```
Looking in indexes: https://pypi.org/simple,
https://www.piwheels.org/simple
Requirement already satisfied: mpi4py in ./.local/lib/python2.7/site-
packages (3.1.5)
```

Demostración rápida de funcionamiento

Un programa "Hola mundo" para MPI en Python, que muestre el rango de cada proceso y el número de procesos ejecutados podría ser así:

```
from mpi4py import MPI

# Inicializa el entorno MPI
comm = MPI.COMM_WORLD

# Obtiene el numero de procesos en el comunicador
size = comm.Get_size()

# Obtiene el rango de este proceso en el comunicador
rank = comm.Get_rank()

# Cada proceso imprime su rango y el numero total de procesos
print (f"Hola desde el proceso {rank} de {size}")
```

Guarde este código en disco como **holampi.py** y ejecútalo así:

```
mpirun -np 4 python3 holampi.py
```

En pantalla aparecerá algo parecido a esto (el orden puede cambiar)

```
Hola desde el proceso 1 de 4
Hola desde el proceso 2 de 4
Hola desde el proceso 3 de 4
Hola desde el proceso 0 de 4
```

Si se usa un número grande de procesos en el parámetro **-np** de **mpirun**, puede aparecer el siguiente error:

```
$ mpirun -np 8 python3 holampi.py

There are not enough slots available in the system to satisfy the 8
slots that were requested by the application:
  python3
Either request fewer slots for your application, or make more slots
available for use.
```

En ese caso, debe usarse el parámetro **–oversubscribe** cuando se invoque **mpirun**, así:

```
$ mpirun --oversubscribe -np 8 python3 holampi.py
Hola desde el proceso 6 de 8
Hola desde el proceso 5 de 8
Hola desde el proceso 1 de 8
Hola desde el proceso 3 de 8
Hola desde el proceso 2 de 8
Hola desde el proceso 4 de 8
Hola desde el proceso 0 de 8
Hola desde el proceso 7 de 8
```

Ejemplo de paso de mensajes entre procesos

Para enviar un valor (un número, una lista, una cadena, etc) a un proceso destino, se emplea la siguiente función. Si llamamos *comm* a la variable que contiene el comunicador, normalmente MPI.COMM_WORLD, entonces:

`comm.send (dato, dest=`*rango_proceso_destino*`, tag=`etiqueta`)`

envía dato al proceso cuyo rango hayamos especificado en *dest* mientras que *tag* indica el valor de la etiqueta que queremos asignar a esta comunicación (un valor numérico cualquiera, por ejemplo 0). Puede omitise.

Para recibir un dato desde un proceso emisor concreto se usa:

`comm.recv (source=`*rango_proceso_emisor*`, tag=`*etiqueta*`)`

La etiqueta debe coincidir con la que se usó en el *send* correspondiente. Si se omitió al enviar, también puede omitirse al recibir.

Ejemplo: un programa que recoge un valor numérico de cada proceso y hace que el proceso 0 los escriba por pantalla.

```
from mpi4py import MPI

# Inicializa el entorno MPI
comm = MPI.COMM_WORLD

# Obtiene el numero de procesos en el comunicador
size = comm.Get_size()

# Obtiene el rango del proceso en el comunicador
rank = comm.Get_rank()

# Todos los procesos envian su numero de rango (valor) al proceso 0
comm.send (rank, dest=0, tag=0)

# Si soy el proceso 0, recibo todos los valores enviados y los
# imprimo en pantalla
if rank == 0:
    for i in range(0, size):
        dato = comm.recv (source=i, tag=0)
        print (f"El proceso {dato} envia un saludo")
```

Este ejemplo asume que *send* nunca se bloquea, lo cual no es cierto (para envíos mayores de 4KiB en esta implementación). Esto puede crear una situación de deadlock en el proceso 0.

Paso de mensajes sin bloqueos

comm.isend (con *i* minúscula) permite enviar un mensaje a un proceso evitando el bloqueo. *isend* vuelve inmediatamente de la llamada devolviendo una **petición**. Es necesario llamar al método **wait** de esa petición para hacer esperar al proceso hasta que realmente se ha terminado el envío. Así, la operación de iniciar el envío y la operación de esperar hasta que finalice pueden hacerse por separado. Así, el ejemplo anterior se puede reescribir de esta manera (en esta ocasión hemos omitido los tags):

```
from mpi4py import MPI

# Inicializa el entorno MPI
comm = MPI.COMM_WORLD

# Obtiene el numero de procesos en el comunicador
size = comm.Get_size()

# Obtiene el rango del proceso en el comunicador
rank = comm.Get_rank()

# isend no bloquea al proceso 0
peticion = comm.isend (rank, dest=0)

# Si somos el proceso 0, recibe todas las respuestas
if rank == 0:
    for i in range(0, size, 1):
        dato = comm.recv (source=i)
        print (f"El proceso {dato} envia un saludo")

# Para todos los procesos, esperamos hasta que el isend anterior ha
# terminado
peticion.wait()
```

Reuniendo todos los valores en un mismo mensaje

comm.gather permite que todos los procesos envíen cada uno un dato a un destinatario, que recogerá una lista con todos esos datos. Esto también e vita la situación de deadlock que teníamos con el primer ejemplo. Se haría así:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

# Gather crea una lista con todos los datos enviados
todos_los_datos = comm.gather (rank, root=0)

# Solo el proceso con rango root tiene acceso a la lista
if rank == 0:
    for dato in todos_los_datos:
        print (f"El proceso {dato} envia un saludo")
```

Suma de dos vectores usando comunicaciones simples

El siguiente programa crea dos vectores con elementos aleatorios y los suma enviando un elemento de cada vector a otro proceso para que éste haga la suma y devuelva el resultado al proceso raíz.

Aquí usamos **tag** para distinguir las dos comunicaciones que hay.

```
from mpi4py import MPI
import random

# cuantos elementos tendrán nuestros vectores
numelementos = 10

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

# Necesitamos tantos procesos como elementos tengamos en los vectores
if numelementos != size:
    if rank == 0:
        print (f"Necesito {numelementos} procesos en mpirun!")
    exit(0)
```

```
# Soy el proceso raíz?
if rank == 0:
    # El proceso raiz crea los vectores a,b con valores aleatorios.
    # Crea también el vector c vacío
    a = [random.randint(-9, 9) for _ in range(numelementos)]
    b = [random.randint(-9, 9) for _ in range(numelementos)]
    c = []
    print (f"Suma de\n{a} y\n{b} igual a\n")

    # Recorremos ambos vectores a,b a la par
    for i in range(size):
        if (i == 0):
            # los primeros elementos los sumamos y añadimos a c
            c.append (a[0] + b[0])
        else:
            # si no, enviamos ambos elementos al proceso i
            comm.send ([a[i], b[i]], dest=i, tag=0)

    # Ahora el proceso raiz recoge las sumas que le envían los
    # procesos y las va añadiendo a c
    for i in range(1,size):
        dato = comm.recv (source=i, tag=1)
        c.append(dato)
    print (c)

else:
    # Estoy aqui si no soy el proceso raíz. Recojo un par de valores
    valores = comm.recv (source=0, tag=0)

    # Los sumo
    resultado = valores[0] + valores[1]

    # Y envio el resultado de vuelta
    comm.send (resultado, dest=0, tag=1)
```

Suma de vectores usando scatter-gather.

comm.scatter trocea los dos vectores y reparte cada par de elementos a cada proceso. Cada uno de ellos hace la suma de esos elementos y los envía de vuelta usando **comm.gather** para que así el proceso raíz obtenga la lista con las sumas de cada elemento.

```
from mpi4py import MPI
import random

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

# Necesitamos tantos procesos como elementos tengamos en los vectores
if numelementos != size:
    if rank == 0:
        print (f"Necesito {numelementos} procesos en mpirun!")
    exit(0)

if rank == 0:
    # Si soy el proceso raíz, creo los vectores a,b con valores aleatorios
    a = [random.randint(-9, 9) for _ in range(numelementos)]
    b = [random.randint(-9, 9) for _ in range(numelementos)]
    print (f"Suma de\n{a} y\n{b} igual a\n")
else:
    # Si no soy el proceso raiz, a,b no tienen nada
    a = None
    b = None

# Cada proceso recibe de parte del root un elemento de a
elemento_a = comm.scatter (a, root=0)

# Cada proceso recibe de parte del root un elemento de b
elemento_b = comm.scatter (b, root=0)

# Cada proceso suma esos dos elementos que ha recibido
elemento_c = elemento_a + elemento_b

# Y los envia con gather para que el root reciba una lista con todos
# ellos
c = comm.gather (elemento_c, root=0)

# El proceso raiz es el único que recibe c, así que lo escribe en
# pantalla
if rank == 0:
    print (c)
```

Suma de vectores usando broadcast

comm.bcast permite enviar el mismo dato a todos los procesos. Lo usaremos para enviar una copia de los vectores *a, b* a todos los procesos. Cada uno de ellos calcula la suma de dos elementos (los que corresponden a su rango) y usamos de nuevo **comm.gather** para recoger en una única lista todas las sumas.

```
from mpi4py import MPI
import random

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

if numelementos != size:
    if rank == 0:
        print (f"Necesito {numelementos} procesos en mpirun!")
    exit(0)

if rank == 0:
    a = [random.randint(-9, 9) for _ in range(10)]
    b = [random.randint(-9, 9) for _ in range(10)]
    print (f"Suma de\n{a} y\n{b} igual a\n")
else:
    a = None
    b = None

# Envio el vector a completo a todos los procesos, root incluido
a = comm.bcast (a, root=0)

# Lo mismo con el vector b
b = comm.bcast (b, root=0)

# Cada proceso suma unicamente los dos elementos que ocupan la
# posición de su rango
elemento_c = a[rank] + b[rank]

# Y los envian con gather de vuelta al proceso raiz
c = comm.gather (elemento_c, root=0)

if rank == 0:
    print (c)
```

Suma de vectores usando comunicaciones simples, con cualquier número de procesos

Usaremos slicing de listas de Python para crear sublistas que enviaremos con **comm.send** a cada proceso que no sea el proceso raíz. Como ahora el número de elementos no tiene por qué ser el mismo que el de procesos, ni siquiera tienen que ser divisibles uno del otro, lo que se hace es:

- Considerar el proceso raíz aparte de los demás. Este proceso se encargará de sumar la sublista que quede restante después de trocear la lista entre los demás procesos
- Así, calculamos el resto de dividir *numelementos* entre *size-1* (todos los procesos menos el raíz). Lo que dé es el número de elementos que sumará el proceso raíz
- El resto de elementos se trocean, ahora sí, en porciones del mismo número de elementos *numelementos // (size-1)*, que se envían al resto de procesos.

```
from mpi4py import MPI
import random

def suma_serie (a, b):
    res = []
    for i in range(len(a)):
        res.append (a[i] + b[i])
    return res
```

NOTA: sí, soy consciente de que la función anterior puede escribirse de esta otra manera:

```
def suma_serie (a, b):
    return [x+y for x,y in zip(a,b)]
```

pero he preferido dejar una versión escrita en "Python para novatos", entre otras cosas, para que se vea mejor la correspondencia con el código C original de la práctica.

```
numelementos = 10

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
```

(sigue…)

```
if rank == 0:
    a = [random.randint(-9, 9) for _ in range(numelementos)]
    b = [random.randint(-9, 9) for _ in range(numelementos)]
    print (f"Suma de\n{a} y\n{b} igual a\n")

    tam_porcion = numelementos // size
    primera_porcion = numelementos - tam_porcion * (size-1)
    inicio_porcion = primera_porcion

    for i in range(1,size):
        porcion_a = a[inicio_porcion : inicio_porcion + tam_porcion]
        porcion_b = b[inicio_porcion : inicio_porcion + tam_porcion]
        inicio_porcion += tam_porcion
        comm.send (porcion_a, dest=i, tag=0)
        comm.send (porcion_b, dest=i, tag=1)

    c = suma_serie (a[:primera_porcion], b[:primera_porcion])

    for i in range(1,size):
        porcion_suma = comm.recv (source=i, tag=2)
        c += porcion_suma

    print (c)

else:
    porcion_a = comm.recv (source=0, tag=0)
    porcion_b = comm.recv (source=0, tag=1)
    porcion_suma = suma_serie (porcion_a, porcion_b)
    comm.send (porcion_suma, dest=0, tag=2)
```

Producto escalar usando comunicaciones colectivas con reducción (al estilo OpenMP)

comm.reduce es parecido a **comm.gather**, pero en vez de reunir muchos datos individuales para devolverlos como una lista, opera con ellos usando una operación de reducción definida, para devolver un único valor. En el siguiente ejemplo se usa para calcular el producto escalar de dos vectores a, b. Usando **comm.scatter** enviamos dos elementos, uno de *a* y otro de *b*, a cada proceso. Dentro del proceso, se multiplican y se envían de nuevo al proceso raíz, pero usando **comm.reduce** e indicando que la operación a realizar es **MPI.SUM**, es decir, sumarlos todos. El resultado lo escribe el proceso raíz en pantalla.

```
from mpi4py import MPI
import random

numelementos = 4

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

if numelementos != size:
    if rank == 0:
        print (f"Necesito {numelementos} procesos en mpirun!")
    exit(0)

if rank == 0:
    a = [random.randint(-9, 9) for _ in range(numelementos)]
    b = [random.randint(-9, 9) for _ in range(numelementos)]
    print (f"Producto escalar de\n{a} y\n{b} igual a\n")
else:
    a = None
    b = None

elemento_a = comm.scatter (a, root=0)
elemento_b = comm.scatter (b, root=0)
producto_parcial = elemento_a * elemento_b
resultado = comm.reduce (product_parcial, root=0, op=MPI.SUM)

if rank == 0:
    print (resultado)
```

Otras funciones

`MPI.Get_processor_name()`

Devuelve una cadena con el nombre del host que alberga el proceso actual.

`MPI.Wtime()`

Mide el tiempo en segundos desde un cierto momento definido en la implementación.

`comm.barrier()`

Suspende cada uno de los procesos en un comunicador a medida que van ejecutando esta función. Cuando todos han entrado en la ejecución de la función, son reactivados de nuevo. Suponemos que *comm* es la variable que tiene el comunicador (normalmente MPI.COMM_WORLD)